

Level Up Coding

★ Member-only story

Claude Code's 5-Layer Agent Development Kit: The Architecture Most Engineers Are Missing

From `CLAUDE.md` and Skills to Hooks, Subagents, and MCP, Claude Code already exposes a layered architecture for memory, expertise, guardrails, delegation, and tool access.



Youssef Hosni

Follow

19 min read · May 6, 2026

👏 332

💬 1

↺ 5

🔖

▶

📄

⋮

Most engineers think they are using **Claude Code** as a coding assistant. In reality, they are sitting on top of a much larger agent runtime that they rarely inspect. Anthropic's own docs show that Claude Code is not just a terminal interface for a strong model.

It already includes persistent memory through **CLAUDE.md**, reusable Skills, deterministic Hooks, delegated Subagents, installable Plugins, and MCP-based connections to external systems. That means the real story is not only what Claude Code can generate, but how its behavior can be shaped, constrained, specialized, and shared across workflows and teams.

This matters because most agent failures are not really prompt failures. They are architectural failures. A system breaks because it has no durable memory layer, no modular knowledge layer, no deterministic guardrail layer, no clean delegation layer, or no way to package behavior for team-wide reuse.

Anthropic's docs now describe each of these mechanisms separately: **CLAUDE.md** for always-on context, **Skills** for on-demand expertise, **Hooks** for workflow automation, **Subagents** for isolated task execution, and **Plugins** for bundling capabilities into a single installable unit.

Once you read the product through that lens, Claude Code starts to look less like "AI in the terminal" and more like an Agent Development Kit hiding in plain sight.

In this article, I break Claude Code down as a five-layer stack: **CLAUDE.md** as the memory layer, **Skills** as the knowledge layer, **Hooks** as the guardrail layer, **Subagents** as the delegation layer, and **Plugins** as the distribution layer. The goal is not just to explain what each piece does, but to show why each one solves a problem that prompting alone cannot solve.

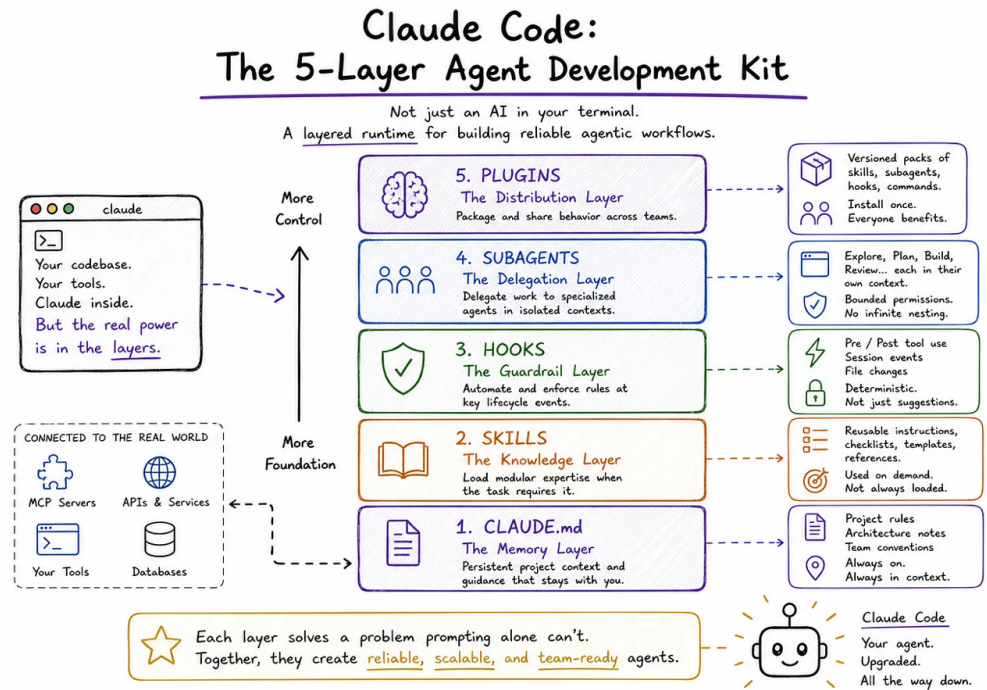


Table of Contents:

1. Claude Code Is More Than a Prompt Interface
2. Layer 1 — CLAUDE.md: The Memory Layer
3. Layer 2 — Skills: The Knowledge Layer
4. Layer 3 — Hooks: The Guardrail Layer
5. Layer 4 — Subagents: The Delegation Layer
6. Layer 5 — Plugins: The Distribution Layer
7. Most Agent Failures Come From a Missing Layer

Enjoying this article? Subscribe to **To Data & Beyond** — a newsletter designed to help you grow in Data Science and AI, beyond the basics.

For a limited time, you can get:

1. 50% To Data & Beyond subscription

2. 50% off my 8 books

3. 50% on all my 6 courses

1. Claude Code Is More Than a Prompt Interface

Most engineers use Claude Code like an unusually capable coding assistant: they open the terminal, give it a task, let it inspect files, run commands, and help move the work forward. That is not wrong, but it is incomplete.

Anthropic's own documentation describes Claude Code as an **agentic**

coding tool that lives in the terminal, understands the codebase, executes routine tasks, and connects to external systems through mechanisms like MCP. Once you read the docs more closely, it becomes clear that Claude Code is not just a model wrapped in a CLI. It is closer to a structured agent runtime with multiple control surfaces for memory, modular knowledge, delegation, deterministic enforcement, and external tool access.

That distinction matters because LLMs alone are not enough for production engineering work. A base model can generate code, explain files, and answer questions, but it does not by itself solve persistent project memory, reusable task expertise, infrastructure-level guardrails, bounded delegation, or standardized team distribution.

Those concerns sit outside the prompt, and Claude Code's architecture increasingly reflects that reality. Anthropic's docs now separately document persistent instruction files like **CLAUDE.md**, extendable Skills, custom subagents, hooks, and preprocessing behavior, settings scopes, plugins, and MCP connectivity. Taken together, these are not random features. They form a layered system for making agent behavior more reliable, reusable, and operationally manageable.

This is why I think it makes sense to view Claude Code through an architectural lens rather than a feature lens. The interesting question is no longer just “**what can Claude Code do?**” The better question is “**what layers does Claude Code expose to control how agent behavior is shaped, constrained, delegated, and shared?**”

Once you look at it that way, the stack becomes much easier to reason about: **CLAUDE.md** acts as the memory and policy layer, **Skills** provide modular expertise, **Hooks** add deterministic guardrails, **Subagents** handle bounded delegation, and **Plugins** plus scoped settings help distribute behavior across teams. MCP and agent-team patterns then sit around this stack as the systems that connect it to the outside world.

That framing also explains why many production failures in agentic workflows do not come from the model being weak. They come from one missing layer.

A team may rely entirely on prompting but have no persistent project memory. Or they may have strong instructions but no deterministic controls around risky tool use. Or they may overload one general agent instead of separating work across specialized subagents. Claude Code is interesting because its docs increasingly expose answers to each of those problems in separate layers. The rest of this article breaks down that stack one layer at a time, starting with the most foundational one: **CLAUDE.md**.

Layer 1 — **CLAUDE.md**: The Memory Layer

The first layer in Claude Code's architecture is **CLAUDE.md**, and it is arguably the most foundational. Anthropic's docs describe **CLAUDE.md** as the place to give Claude **persistent instructions** about how to work in a project, while auto memory stores notes Claude writes for itself based on repeated corrections and preferences.

That distinction matters. **CLAUDE.md** is not a scratchpad for temporary

prompts. It is the layer where teams encode durable context: coding standards, architectural decisions, preferred libraries, review checklists, and workflow constraints that should be present from the start of every session.

The important technical point is that Claude Code does not treat **CLAUDE.md** as a single project file loaded only from the current directory. According to the docs, Claude walks **up the directory tree** from the working directory and loads every **CLAUDE.md** and **CLAUDE.local.md** it discovers along the way.

These files are concatenated into context rather than overriding each other, with instructions closer to the launch directory read later.

This is a much more flexible model than most engineers assume. It means **CLAUDE.md** is not just “one config file,” but a scoped memory system that can express broad rules at higher levels and more specific rules nearer to the active code.

Claude Code — MEMORY.md: Everything you need to know & how to get started?

Read the full article for free through this friend link.

levelup.gitconnected.com

That loading model is what makes **CLAUDE.md** useful as a real memory layer instead of just another instruction file. A top-level file can capture global repository constraints such as naming conventions, test expectations, or architectural boundaries, while a deeper file can narrow the behavior for one subsystem.

Anthropic also supports imported instruction files through **@path/to/import** syntax, which allows teams to split large instruction sets into smaller reusable modules rather than turning one markdown

file into a dumping ground. Imported files are expanded into context at launch, and imports can chain recursively up to five hops. In practice, this makes **CLAUDE.md** closer to a composable project memory system than to a static prompt template.

There is also an important operational distinction between **CLAUDE.md** and auto memory. Anthropic's docs explicitly say that both are loaded at the start of every conversation, but they serve different roles.

CLAUDE.md contains the instructions humans intentionally write. Auto memory stores Claude's own accumulated notes in a machine-local memory directory, indexed through **MEMORY.md**, with only the first 200 lines or 25 KB of that index loaded at session start.

By contrast, **CLAUDE.md** files are loaded in full regardless of length, although Anthropic notes that shorter files tend to produce better adherence. So **CLAUDE.md** is the controlled, declarative memory layer, while auto memory is the adaptive, emergent one.

*This is why I think the right way to interpret **CLAUDE.md** is not as “context you paste once so Claude remembers your style.” It is more like the agent's working constitution.*

Anthropic's docs even note organization-wide deployment paths for centrally managed **CLAUDE.md** files on macOS, Linux, WSL, and Windows, which reinforces that this layer is meant to carry durable operational policy, not only personal preferences.

Once you look at it that way, the role of **CLAUDE.md** becomes much clearer: it is the memory and policy layer that keeps teams from restating the same constraints in every session and gives the agent a stable behavioral baseline before any task-specific work begins.

Layer 2 — Skills: The Knowledge Layer

If **CLAUDE.md** is the persistent memory and policy layer, then **Skills** are the layer that gives Claude Code modular expertise on demand.

Anthropic's documentation describes Skills as a way to extend what Claude can do by placing instructions in a **SKILL.md** file.

Claude can invoke a skill directly through a slash command or load it automatically when it detects that the task is relevant. The important architectural point is that Skills are not loaded like permanent project memory. Unlike **CLAUDE.md**, a skill's body is only brought into context when it is actually needed, which makes it a much cleaner way to attach reusable procedures, checklists, or domain-specific guidance without bloating every session by default.

That distinction matters more than it first appears. A lot of engineers treat agent customization as one long prompt: put everything into a persistent context and hope the model figures out what matters.

Skills offer a more structured alternative. Anthropic explicitly recommends creating a skill when you find yourself repeatedly pasting the same instructions, or when part of **CLAUDE.md** has grown into a procedure rather than a fact. In other words, **CLAUDE.md** should hold durable project rules, while Skills should hold reusable operational know-how. That separation keeps the always-loaded context lean and turns complex workflows into modular components Claude can reach for when appropriate.

Anthropic's docs also make clear that Skills are not just renamed custom commands. The platform now treats older **.claude/commands/*.md** files and newer **.claude/skills/*/SKILL.md** definitions as functionally similar at the invocation level, but Skills add more control.

They support a directory structure with supporting files, invocation control so either the user or Claude can trigger them, dynamic context injection, and even execution in a subagent. That is why I think calling

Skills the “**knowledge layer**” is the right framing. They are not just shortcuts. They are the unit by which expertise, procedures, and reusable task context become composable inside the agent runtime.

This also connects directly to your architectural diagram. In practice, a Skill works like a bounded packet of task-specific knowledge: a description tells Claude what the skill is for, runtime matching decides whether it applies, and the relevant instructions are loaded only when useful. Anthropic's docs explicitly note that bundled skills such as `/debug`, `/loop`, and `/simplify` are prompt-based rather than fixed logic, which is an important clue about the overall design.

Skills are the mechanism by which Claude's behavior becomes modular without requiring everything to be hard-coded into the core product. That is a much more scalable pattern than expecting a single giant base prompt to cover every workflow.

The practical value is straightforward. Suppose a team has a standard process for debugging flaky tests, reviewing SQL migrations, or preparing release notes. Those are all cases where persistent memory alone is too broad and one-off prompting is too repetitive.

A Skill lets the team package the procedure once, keep reference material alongside it, and let Claude load that knowledge only when the task calls for it. Anthropic even notes that long reference material inside Skills costs almost nothing until used, which means Skills are not just cleaner architecturally, but also more efficient operationally.

*This is why I see Skills as the second essential layer in Claude Code's stack. **CLAUDE.md** gives the agent a stable constitution. Skills give it targeted, reusable expertise without inflating the main context window.*

You can read more and get started with Skills, with my comprehensive guide: [Claude Code Skills 101: Everything You Need to Get Started With](#)

Claude Code Skills 101: Everything You Need to Get Started With

If you have been using Claude Code, you have probably noticed that every new session starts the same way: you...

levelup.gitconnected.com

That combination is what turns Claude Code from a general assistant into a system that can carry both durable project memory and modular domain knowledge at the same time. The next layer is what makes that behavior safer in practice: Hooks.

Layer 3 — Hooks: The Guardrail Layer

If **Skills** are the modular knowledge layer, then **Hooks** are the layer that turns agent behavior into something more operationally enforceable. Anthropic's official docs describe **Hooks** as a way to automate workflows around **Claude Code** by reacting to events and running commands before or after certain actions.

The **hooks** guide explicitly covers patterns such as auto-formatting code after edits, blocking edits to protected files, re-injecting context after compaction, auditing configuration changes, reloading environment state when files change, and auto-approving specific permission prompts. That is a very different role from prompting. A prompt can suggest behavior. A hook can deterministically intercept or transform it.

This is why I think **Hooks** are best understood as the guardrail layer rather than just an automation convenience. Anthropic's docs show that hooks can be attached to events such as **PreToolUse**, and the cost guide gives a concrete example where a **PreToolUse** hook intercepts Bash commands and rewrites test invocations so Claude sees only failing output instead of the full stream.

In that example, the hook reads the incoming JSON payload, inspects the command, and returns structured JSON that updates the tool input. That is not “AI deciding better.” It is infrastructure controlling what the model is allowed to see or do at a given point in the workflow.

That distinction matters because prompt-level guardrails are often too soft for production use. If a team wants to ensure that generated code is always reformatted, that sensitive files cannot be changed, or that destructive tool invocations require stricter review, relying on instructions alone is fragile. Hooks move those controls outside the model's discretion.

The hooks docs explicitly frame them around automation patterns like blocking edits, notifying on specific lifecycle events, and keeping session context aligned after compaction. In other words, Hooks make it possible to enforce workflow behavior at the event level rather than merely asking the model to behave.

Hooks also fit neatly into the broader Claude Code architecture because they operate alongside context management rather than inside it. Anthropic's troubleshooting docs even list "hooks not firing" as a distinct configuration issue, which is a useful clue about how the product treats them: **Hooks** are part of the runtime environment, not just content inside the chat.

The docs also recommend `/doctor` for diagnosing issues with hooks, MCP servers, and context usage, which reinforces that Hooks belong to the execution layer of the system. They are not just instructions Claude reads; they are configuration-driven behavior that must be wired correctly to the session runtime.

There is also a practical cost and performance angle here. Anthropic's cost guide explicitly recommends offloading processing to hooks and skills because hooks can preprocess large inputs before Claude sees them. Their own example is filtering a huge test log down to only failures, reducing what would have been tens of thousands of tokens to a much smaller payload.

That is an important architectural lesson: **Hooks** are not only about safety. They are also about efficiency. A well-placed hook can reduce token waste, keep context cleaner, and prevent the model from spending time on predictable filtering work that deterministic shell logic can do more cheaply and more reliably.

This is why I think Hooks are the layer most teams underestimate first. Persistent memory helps the agent remember. Skills help it specialize. But Hooks are what let a team convert repeated operational expectations into deterministic policy.

They sit at the boundary between model-driven reasoning and system-enforced behavior, which is exactly where many production failures actually happen. Once that layer is missing, quality depends too heavily

on the prompt. The next layer, Subagents, solves a different problem: not control, but delegation.

Layer 4 — Subagents: The Delegation Layer

If Hooks solve the problem of control, Subagents solve the problem of delegation. Anthropic's docs describe subagents as **specialized AI assistants** that handle specific types of tasks in their own context window and return only the result.

That design matters because one of the fastest ways to degrade an agent session is to let every side task pollute the main conversation with logs, search results, or exploratory reasoning that will never be reused.

Subagents are Claude Code's answer to that problem: instead of forcing one general agent to do everything in one thread, the main agent can delegate a side task to a specialized worker that operates independently and reports back a summary.

Subagents help preserve context by keeping exploration and implementation out of the main conversation, enforce constraints through limited tool access, reuse configurations across projects, specialize behavior through focused prompts, and even reduce cost by

routing suitable tasks to cheaper models like Haiku.

That is why I think “**delegation layer**” is the right framing. A **subagent** is not just a named prompt. It is a bounded execution unit with its own description, system prompt, model, tool permissions, memory settings, and runtime scope.

This also explains why subagents are more than just a convenience for power users. Anthropic includes several built-in subagents such as **Explore**, **Plan**, and **General-purpose**, each with different tool access and intended use.

Explore is optimized for read-only search and codebase analysis, **Plan** is used during planning workflows, and the **general-purpose** agent handles more complex multi-step tasks that involve both reasoning and action.

Importantly, the docs also state that built-in subagents inherit the parent conversation's permissions with additional tool restrictions. That is a subtle but important systems choice: delegation is not unrestricted. It is bounded by design.

The docs also confirm a key point from your social post: **subagents** do **not** recurse infinitely. Anthropic explicitly notes that subagents cannot spawn other subagents, and that this restriction is part of how Claude avoids infinite nesting while still gathering necessary context.

This is a crucial design decision. Without a hard boundary like that, delegation can easily turn into uncontrolled agent trees that are expensive, hard to monitor, and difficult to reason about. By preventing nested spawning, Claude Code keeps the delegation model simpler: the main agent delegates work down, the subagent performs it in isolation, and the result comes back up.

From a practical engineering perspective, the most useful part of Anthropic's design is that subagents are configurable across multiple scopes.

They can be created interactively through the `/agents` command, stored at the user level in `~/.claude/agents/`, stored at the project level in `.claude/agents/`, passed by CLI for the current session, or distributed through plugins.

Each subagent is just a Markdown file with **YAML frontmatter**, but the behavior it defines is rich: model choice, tool access, memory behavior, hooks, and skills can all be scoped to that specific delegated worker. In other words, subagents are not simply role labels. They are packaged execution profiles for specialized work.

This is why I think subagents are one of the most important architectural layers in Claude Code.

CLAUDE.md keeps the main agent grounded in project rules. Skills give it modular expertise. Hooks enforce policy at runtime. But **subagents** are what prevent the whole system from collapsing into one overloaded conversation with too many responsibilities. They create hard boundaries around side work, let teams specialize workflows, and keep the main context cleaner and more durable.

Once you have that delegation layer, the next problem becomes organizational rather than technical: how do you package and distribute all of this behavior across a team? That is where the next layer comes in.

Layer 5 — Plugins: The Distribution Layer

Once a team has persistent memory in **CLAUDE.md**, modular expertise in **Skills**, deterministic controls through **Hooks**, and bounded delegation via **Subagents**, the next problem is no longer local behavior. It is **distribution**.

How do you make that architecture portable across repos, machines, and teammates without rebuilding it by hand every time? This is where the plugin layer becomes useful as a mental model.

Anthropic's docs expose multiple mechanisms for packaging and reusing Claude Code behavior across scopes: user-level and project-level subagents, shared Skills, plugin settings, organization-wide **CLAUDE.md** deployment, and even plugins that ship code intelligence or additional capabilities.

In practice, that means Claude Code is not just configurable for one

person. It is increasingly designed to let teams standardize and distribute how the agent behaves.

The strongest official signal here comes from Anthropic's plugin and settings model. The costs and settings docs describe dedicated plugin configuration sections, plugin allowlists, and plugin permissions such as **codeIntelligence.***, with support for project-local settings and inherited user settings.

Anthropic also notes that CLI arguments like — **append-system-prompt** and — **allowedTools** do not propagate into plugins, which reinforces an important architectural point: plugins are treated as bounded distribution units with their own behavior surface, not as passive extensions of the parent session.

That is exactly why I think “distribution layer” is the right framing. Plugins are not just about adding tools. They are about packaging agent behavior so it can be installed, reused, and governed at scale.

This same distribution pattern shows up in subagents and Skills as well. Anthropic's subagent docs support multiple installation scopes, including user-level storage in `~/.claude/agents/`, project-level storage in `.claude/agents/`, session-only subagents passed by CLI, and plugin-defined subagents that can be distributed as part of a broader package.

Skills follow a similarly modular design, allowing reusable operational knowledge to live outside the always-loaded memory layer and be invoked only when relevant. Put differently, Claude Code's architecture does not stop at “**can the agent do this?**” It increasingly asks “**how can this capability be packaged and shared?**” That is the hallmark of a real platform rather than a one-user assistant.

There is also a governance angle to this layer. The plugin model lets teams distribute not only convenience, but policy. A shared package can

carry approved subagents, carefully scoped tool permissions, standard hooks, or organization-specific workflows.

Anthropic's memory docs even document centrally managed **CLAUDE.md** paths for enterprise deployment across macOS, Linux, WSL, and Windows, which shows that standardization is not an afterthought.

The same layered architecture that helps an individual engineer work faster can also be turned into a team-wide operating model, where memory, expertise, controls, and delegation are installed once and inherited consistently. That is a much more robust model than relying on everyone to remember the same prompt recipes.

This is why I think **Plugins** are the final core layer in the stack. **CLAUDE.md** defines stable rules. **Skills** package expertise. **Hooks** enforce runtime policy. **Subagents** create bounded workers. **Plugins** and shared packaging mechanisms make all of that portable.

They are what let agent behavior move from a personal setup into a team asset. And once that distribution layer exists, the whole stack becomes much easier to connect to the outside world through MCP servers and to compose into broader agent-team patterns.

Most Agent Failures Come From a Missing Layer

What makes Claude Code interesting is not just that it is an agentic coding tool. Anthropic's docs make clear that it is also an extensible runtime with distinct mechanisms for persistent instructions, modular expertise, deterministic workflow control, delegated execution, and external tool integration.

CLAUDE.md defines durable project guidance, **Skills** load reusable task knowledge when relevant, **Hooks** execute automatically at specific lifecycle points, and **Subagents** isolate focused subtasks in separate contexts. Taken together, these are not isolated features. They are

separate answers to separate failure modes in real agent systems.

That is why I think most production failures in agentic workflows are better understood as **architectural failures** than model failures. When a system forgets team conventions, the missing layer is usually persistent memory. When it keeps repeating the same setup instructions, the missing layer is modular knowledge.

When it behaves inconsistently around risky actions, the missing layer is deterministic guardrails. When the main context becomes noisy and overloaded, the missing layer is delegation. And when behavior cannot be standardized across projects or teams, the missing layer is distribution. The model may still be good, but without the surrounding layers, the workflow stays fragile.

Anthropic's own guidance reinforces this layered reading. Their best-practices docs explicitly distinguish advisory instructions from deterministic hooks, noting that hooks are the right tool when something must happen every time with zero exceptions.

Their subagent docs emphasize context preservation and specialization. Their skills docs frame skills as the right place for repeated instructions, checklists, and multi-step procedures that should not live in always-loaded memory.

Even the overview pages consistently describe Claude Code as a tool that reads codebases, runs commands, and integrates with development environments, which is much closer to an operational system than to a chat interface.

My take is that this is the more useful way to think about Claude Code going forward. The biggest leap is not that the model got better at coding. It is that the surrounding system is increasingly exposing the layers needed to make coding agents more reliable in practice.

Once you stop viewing Claude Code as “just an assistant in the terminal,” the architecture becomes easier to reason about. `CLAUDE.md` gives the agent a constitution. Skills give it modular expertise. Hooks enforce quality at runtime. Subagents keep delegation bounded. MCP and the broader extension surface connect the stack to real tools and real workflows.

And that, to me, is the real Agent Development Kit hidden inside Claude Code. Not one magic feature, but a layered system where each part solves a problem that prompting alone cannot solve.

Join my upcoming live workshop: Designing Multi-Agent Deep Search Systems

If you want to understand how to architect deep search agents beyond simple retrieval and basic tool calling, I’m running a live technical workshop on this topic.

We’ll cover the full architecture: planner agents, executor agents, tool design, MCPs, source reliability scoring, contradiction handling, temporal reasoning, memory management, merging, and evaluation.

You’ll also get the recording, slide deck, technical handbook, architecture blueprint, source scoring template, and evaluation checklist.

| *Book Your Seat*

Claude Code

Claude Skills

LLM

AI

Youssef Hosni



Published in Level Up Coding

Following

329K followers · Last published 1 day ago

Coding tutorials and news. The developer homepage [gitconnected.com](#) && [skilled.dev](#) && [levelup.dev](#)



Written by Youssef Hosni

Follow

32K followers · 95 following

Master Claude Code: <https://todatabeyond.substack.com/subscribe>

